Chapter 4

堆疊和佇列

4.1 前言

- 堆疊 (Stack) 結構
  - 只有單一的出入口，我們稱之為頂端 (Top)。
  - 資料的「新增」和「刪除」都從堆疊的頂端進出。
  - 後進入堆疊的資料反而先出來。
- 佇列 (Queue) 結構
  - 有一個入口和一個出口，我們稱入口為尾端 (Rear)，出口為頭端 (Front)(或稱前端)。
  - 資料從尾端「新增」，從頭端「刪除」。
  - 先進入佇列的資料將先出來。

4.2 堆疊 (Stack)

- 堆疊具有以下特徵：
  - 堆疊是一個有順序性的資料結構。
  - 資料的「新增」和「刪除」都從堆疊的頂端 (Top) 這個唯一的出入口。
  - 具有「後進先出 (Last In First Out ： LIFO) 」(或「先進後出」 (First In Last Out ： FILO)) 的特性。

4.2 堆疊 (Stack)

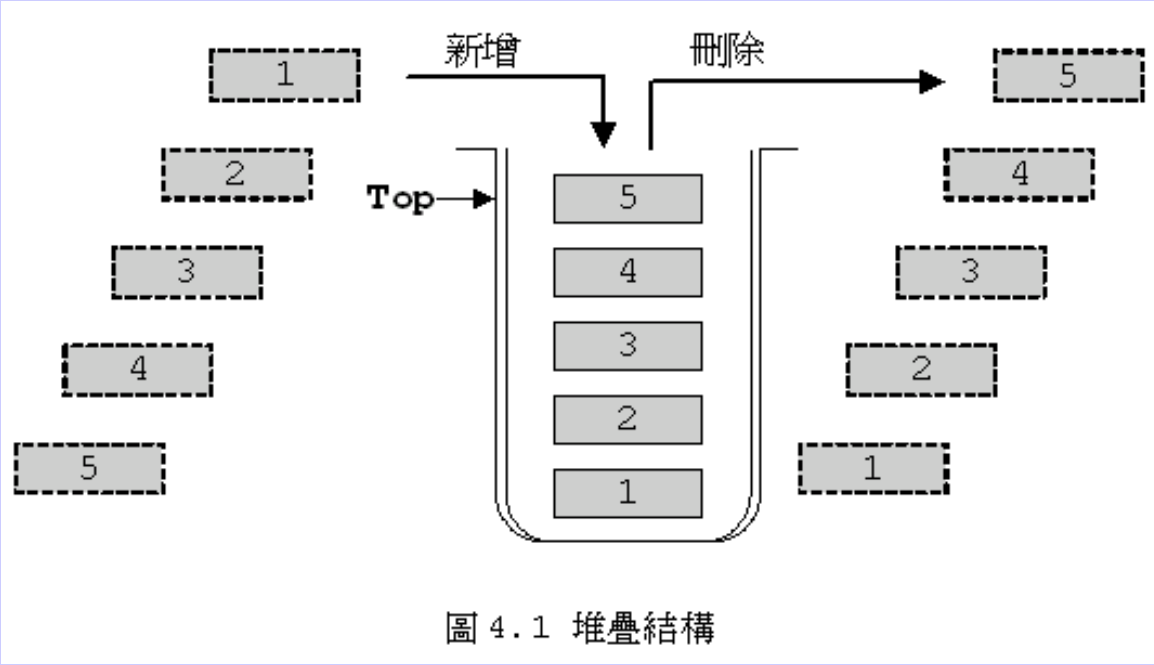


圖 4.1 堆疊結構

4.2.1 以陣列實作堆疊

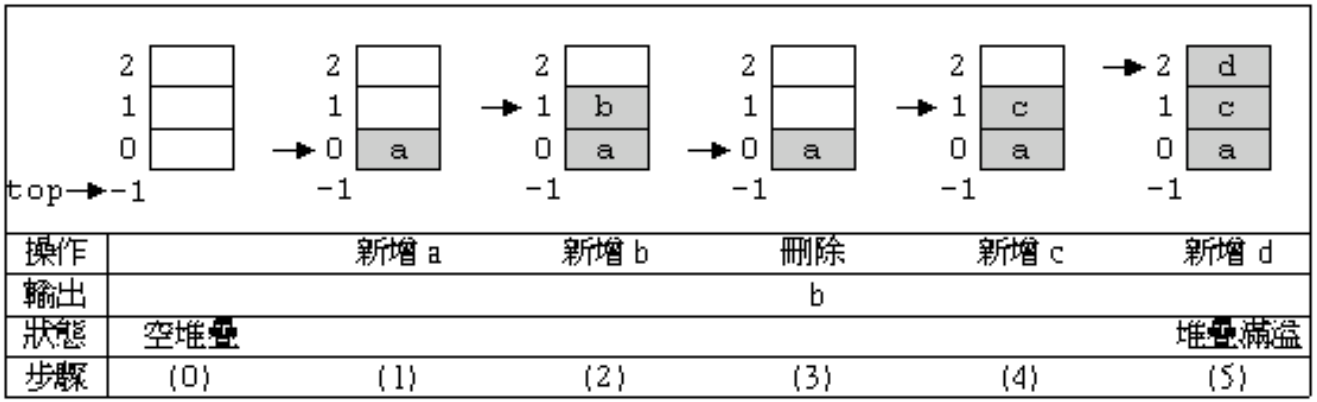
- 以「陣列」實作堆疊時可歸納成底下幾個基本操作：
  1. 宣告陣列 stack[n] ：
     陣列大小 n 即代表堆疊容器的容量，且索引編號為 0、 1、 2、 ...、 n-1。
  2. 設定堆疊的頂端指標 top ：
     一開始 top=-1 ，表示堆疊裡空無一物。
     當 top=n-1 時，表示堆疊已滿，無法再容納資料了。
  3. 判斷堆疊是否已滿：
     當 top = n-1 時，表示堆疊已滿。

4.2.1 以陣列實作堆疊

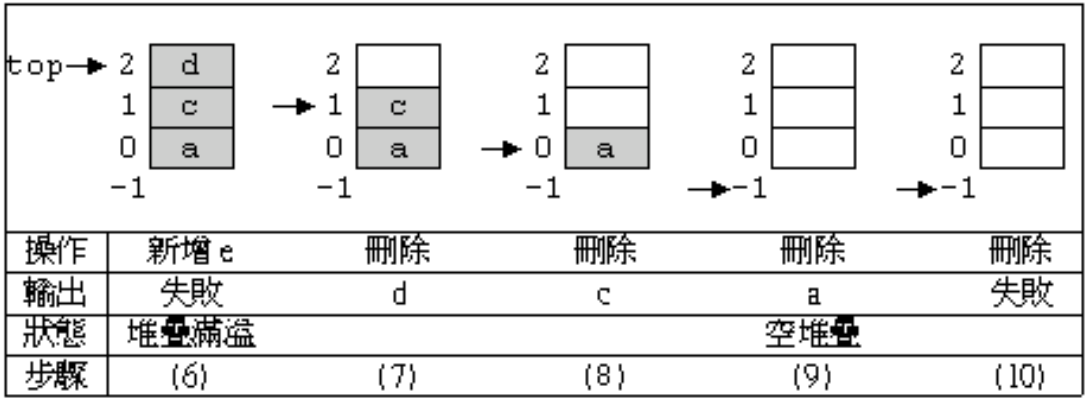
4. 判斷堆疊是否已空：
   當 top < 0 時，表示堆疊已空。
5. 新增資料：
   將資料放入堆疊的動作我們稱之為 push。若堆疊未滿，則 top = top + 1 後再將資料存入 stack [top] 位置。
6. 刪除資料：
   從堆疊頂端取出資料的動作我們稱之為 pop。若非空堆疊，則自堆疊頂端將 stack [top] 資料取出，然後 top = top - 1。

4.2.1 以陣列實作堆疊

【例 1 】以陣列來實作堆疊的新增和刪除



操作		新增 a	新增 b	刪除	新增 c	新增 d
輸出				b		
狀態	空堆疊					堆疊滿溢
步驟	(0)	(1)	(2)	(3)	(4)	(5)



操作	新增 e	刪除	刪除	刪除	刪除
輸出	失敗	d	c	a	失敗
狀態	堆疊滿溢			空堆疊	
步驟	(6)	(7)	(8)	(9)	(10)

圖 4.2 以陣列 stack[3]實作堆疊的新增和刪除動作

4.2.2 以鏈結串列實作堆疊

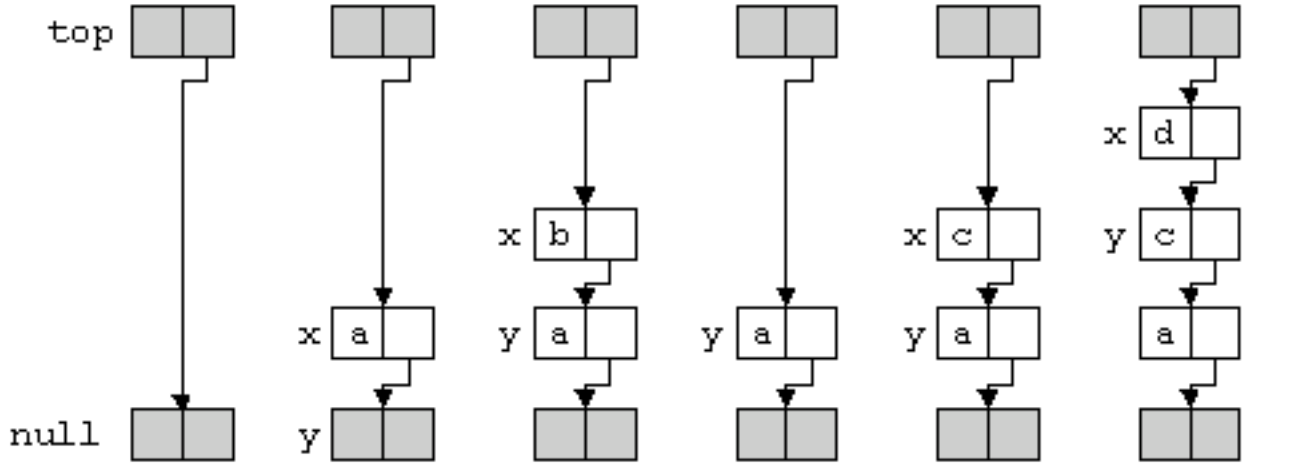
- 以「鏈結串列」實作堆疊時可歸納成底下幾個基本操作：
  1. 建立堆疊結構：
     宣告一個鏈結串列結構，一開始只有一個名為 top 的頭節點，它的鏈結欄指向 null。
  2. 判斷堆疊是否已空：
     當 top 節點指向 null 時，表示堆疊已空無一物。

4.2.2 以鏈結串列實作堆疊

3. 新增資料：
   將資料放入堆疊的動作我們稱之為 push。首先，向記憶體索取一個節點空間 x ，然後將要新增的資料存入節點 x 的資料欄。接下來令 top 所指的節點為 y ，將 x 指向 y ，再將 top 指向 x。
4. 刪除資料：
   從堆疊頂端取出資料的動作我們稱之為 pop。若非空堆疊，則令 top 所指之節點為 x ，且 x 所指的節點為 y。輸出 x 的資料欄，更改 top 鏈結指標值，將它指向 y。

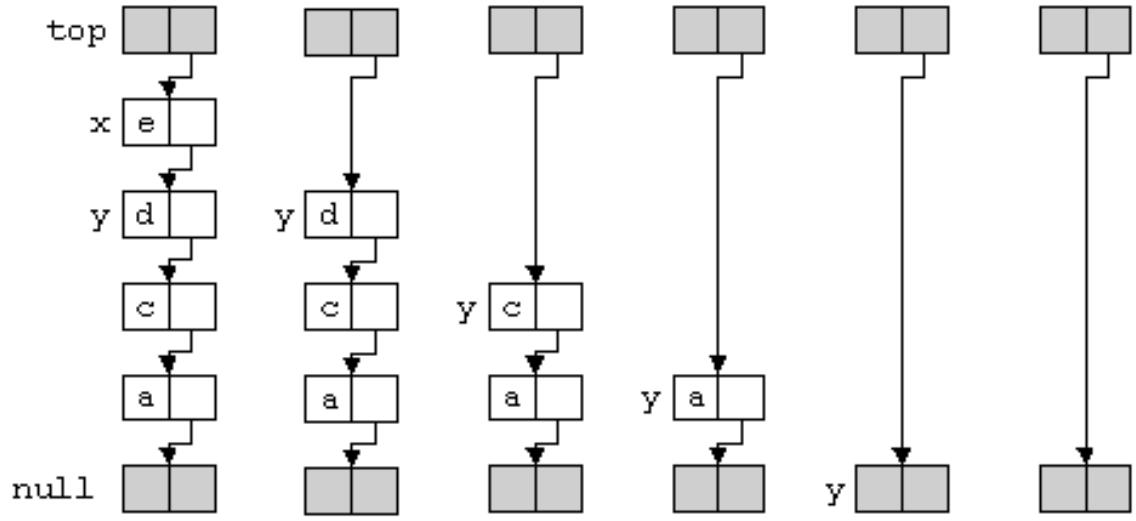
4.2.2 以鏈結串列實作堆疊

【例 2 】以鏈結串列來實作堆疊的新增和刪除



操作		新增 a	新增 b	刪除	新增 c	新增 d
輸出				b		
狀態	空堆疊					
步驟	(0)	(1)	(2)	(3)	(4)	(5)

4.2.2 以鏈結串列實作堆疊



操作	新增 e	刪除	刪除	刪除	刪除	刪除
輸出		e	d	c	a	
狀態					空堆疊	失敗
步驟	(6)	(7)	(8)	(9)	(10)	(11)

圖 4.3 以鏈結串列實作堆疊的新增和刪除動作

4.3 堆疊的應用

- 4.3.1 副程式的呼叫和返回
- 4.3.2 河內塔 (Towers of Hanoi)
- 4.3.3 運算式的轉換
- 4.3.4 運算式的求值

4.3.1 副程式的呼叫和返回

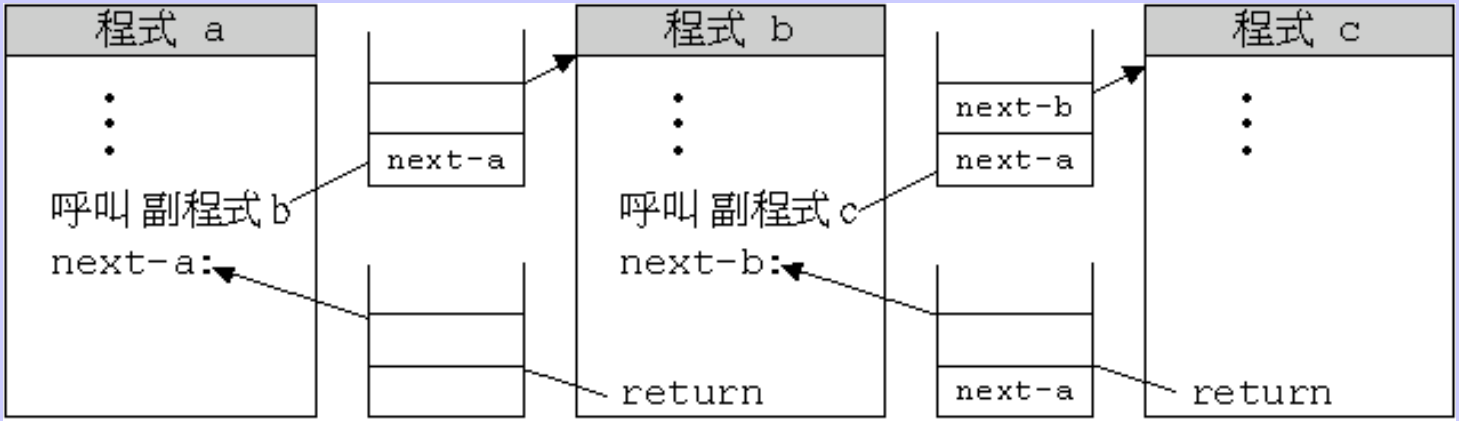


圖 4.4 副程式的呼叫和返回

4.3.2 河內塔 (Towers of Hanoi)

- 河內塔問題是一個非常有趣的智力遊戲，問題描述如下：
  - 假設有編號分別為 a、 b、 c 的三根柱子，一開始在 a 柱上放置了 n 個中間有孔的圓盤，圓盤的編號由上到下分別為 1 ，2 ，…，n ，且編號愈大代表圓盤的直徑也愈大。
  - 所謂河內塔問題乃是要將所有 n 個圓盤從 a 柱搬移到 c 柱，且在圓盤搬移的過程中必須遵守下列 3 條規則：
    1. 無論何時，每一根柱子上直徑較小的圓盤都要放在直徑較大的圓盤的上面，亦即須保持「上小下大」的排列方式。
    2. 每次僅能移動一個圓盤。
    3. 圓盤可以在 3 根柱子間任意的移動。

4.3.2 河內塔 (Towers of Hanoi)

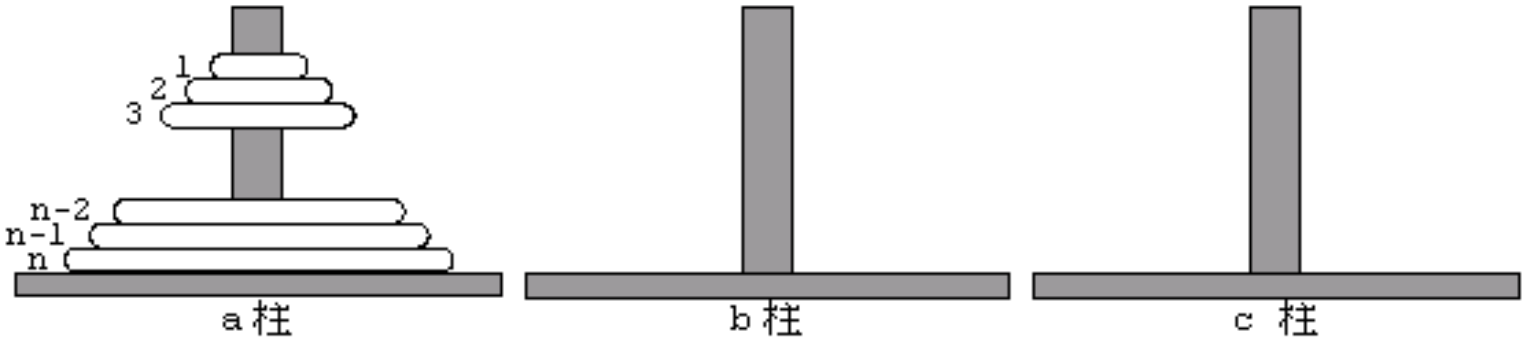


圖 4.5 河內塔(搬移前)

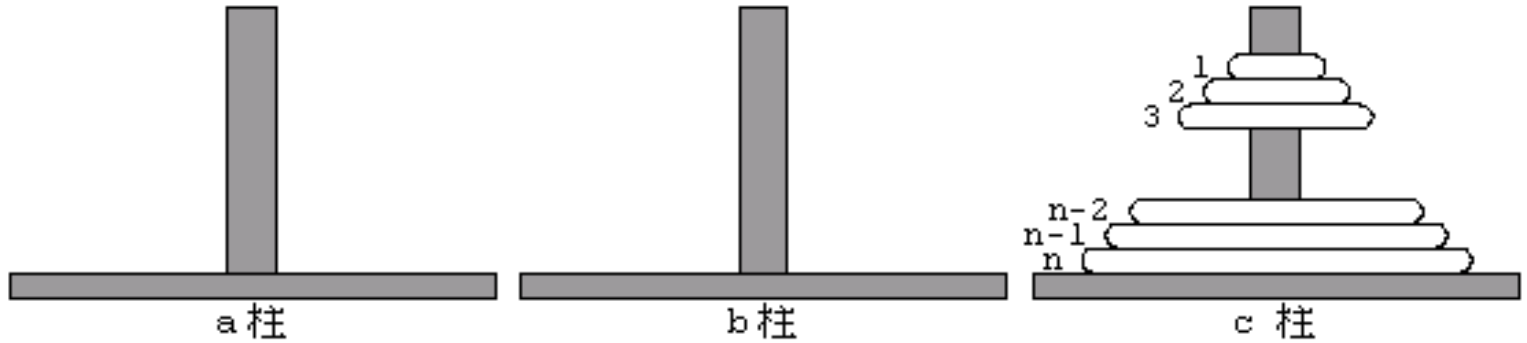


圖 4.6 河內塔(完成後)

4.3.2 河內塔 (Towers of Hanoi)

- 當 n = 2(有 2 個圓盤) 時，一共需 3 次搬動，搬動的順序如下：
  - 步驟 1 ：將 1 號圓盤從 a 柱搬到 b 柱。
  - 步驟 2 ：將 2 號圓盤從 a 柱搬到 c 柱。
  - 步驟 3 ：將 1 號圓盤從 b 柱搬到 c 柱。

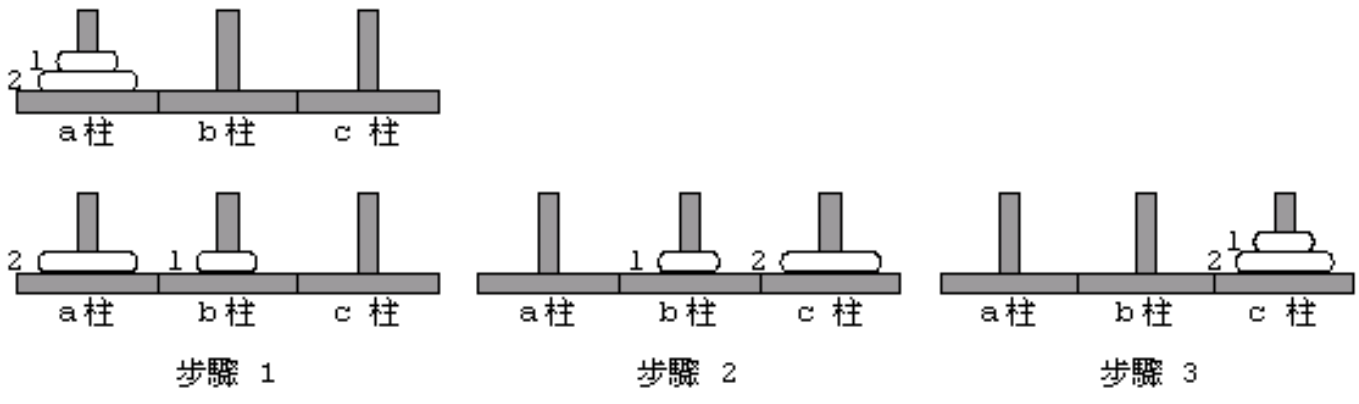


圖 4.7 河內塔當 n=2 時的搬移過程

4.3.2 河內塔 (Towers of Hanoi)

- 當 n = 3(有 3 個圓盤) 時，一共需 7 次搬動，搬動的順序如下：
  - 步驟 1 ：將 1 號圓盤從 a 柱搬到 c 柱。
  - 步驟 2 ：將 2 號圓盤從 a 柱搬到 b 柱。
  - 步驟 3 ：將 1 號圓盤從 c 柱搬到 b 柱。
  - 步驟 4 ：將 3 號圓盤從 a 柱搬到 c 柱。
  - 步驟 5 ：將 1 號圓盤從 b 柱搬到 a 柱。
  - 步驟 6 ：將 2 號圓盤從 b 柱搬到 c 柱。
  - 步驟 7 ：將 1 號圓盤從 a 柱搬到 c 柱。

4.3.2 河內塔 (Towers of Hanoi)

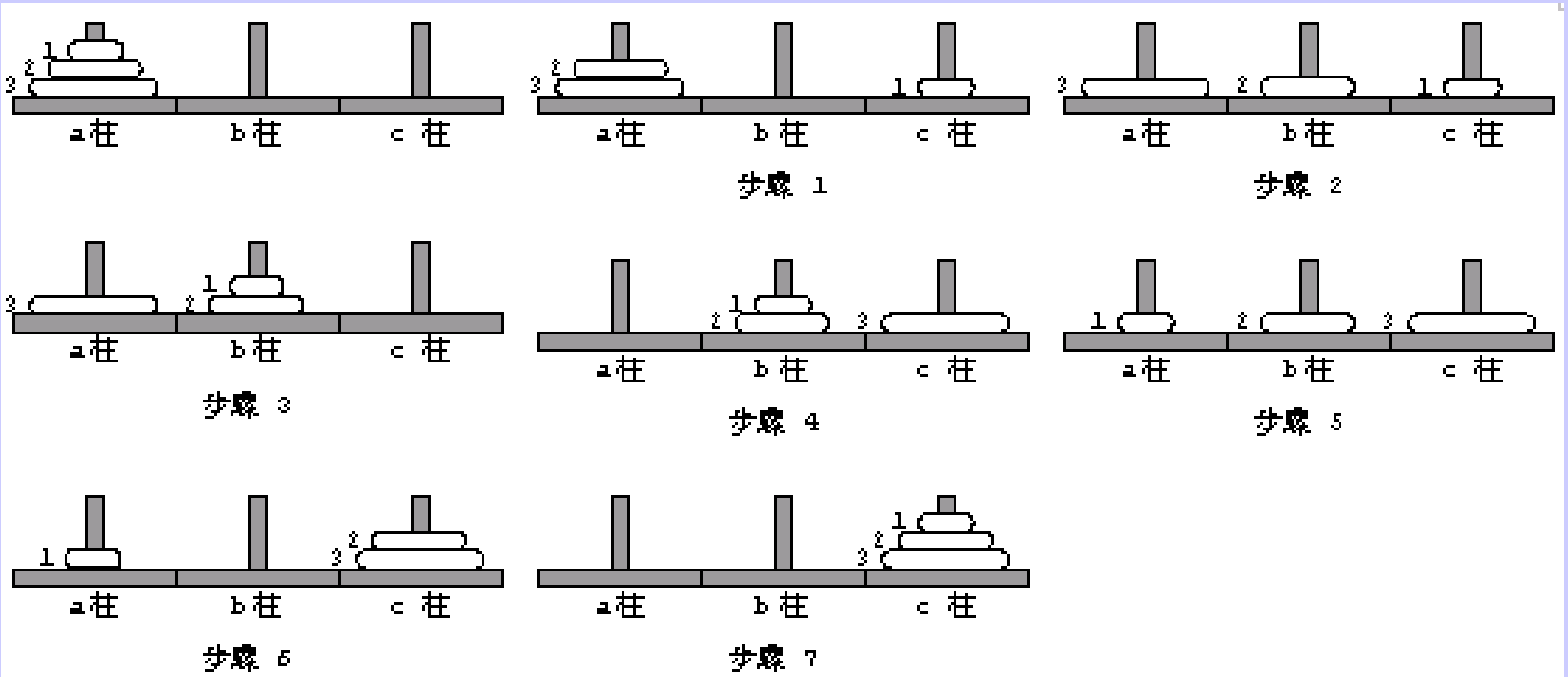


圖 4.8 河內塔當 n=3 時的搬移過程

4.3.3 運算式的轉換

- 「 a+b/(c-d)*e 」是一個運算式 (Expression)
  - 運算元 (Operand) ： a、 b、 c、 d、 e。
  - 運算子 (Operator) ： +、 -、 *、 /。
- 運算式求值必須遵守下列規則：
  1. 括號內的運算式須優先處理。
  2. 由左至右。
  3. 依運算子的優先順序處理，例如先乘除後加減等。

4.3.3 運算式的轉換

表 4.1 運算子的種類及優先順序

運算子種類		符號種類	優先順序值
(1)	前置運算子	正號、負號	8
(2)	算術運算子	^(次方)	7
		*、/	6
		+、-	5
(3)	關係運算子	<、<=、!=、=、>=、>	4
(4)	邏輯運算子	!(not/非)	3
		&&(and/且)	2
		||(or/或)	1

4.3.3 運算式的轉換

- 運算式的表示法：
  1. 前置 (Prefix) 運算式：

     運算子放在兩個運算元前面，沒有括號，不 易閱讀。
  2. 中置 (Infix) 運算式：

     運算子放在兩個運算元之間，可以有括號，最容易閱讀。
  3. 後置 (Postfix) 運算式：

     運算子是放在兩個運算元之後，沒有括號，不易閱讀。

4.3.3 運算式的轉換

- 運算式的表示法：

表 4.2 運算式的三種表示法

運算式表示法		格式	範例
1	前置式	<運算子><運算元><運算元>	-+a*/b-cdef
2	中置式	<運算元><運算子><運算元>	a+b/(c-d)*e
		<運算子><運算元>	-a
3	後置式	<運算元><運算元><運算子>	abcd-/e*+f-

4.3.3 運算式的轉換

- 中置運算式轉換成前置運算式：
  1. 依據運算式的求值規則，先用括號將相關的運算組合括起來。
  2. 將運算子取代左邊最接近的左括號（。
  3. 移除所有的右括號)。

4.3.3 運算式的轉換

- 中置運算式轉換成後置運算式：
  1. 依據運算式的求值規則，先用括號將相關的運算組合括起來。
  2. 將運算子取代右邊最接近的右括號)。
  3. 移除所有左括號 (。

4.3.3 運算式的轉換

【例１】中置運算式轉成前置運算式

將中置運算式 **a+b/(c-d)*e-f** 轉成對應的前置運算式之步驟如下：

1.((a+((b/(c-d))*e))-f)

2.-+a*/b-cd))e))f)

3.-+a*/b-cdef

4.3.3 運算式的轉換

【例２】中置運算式轉成後置運算式

將中置運算式 **a+b/(c-d)*e-f** 轉成對應的後置運算式之步驟如下：

1.((a+((b/(c-d))*e))-f)

2.((a((b(cd-/e*+f-

3.abcd-/e*+f-

4.3.3 運算式的轉換

【演算邏輯】利用堆疊將中置運算式轉換成後置運算式

步驟 1：從左到右依次讀取中置運算式的每一個元素。

```
x = 讀入的元素;
switch(x){
  case('(') : 將 x 放入堆疊;
              break;
  case(')') : x = 自堆疊頂端彈出的運算子;
              while(x != '('){
                 將 x 輸出至後置式;
                 x = 自堆疊頂端彈出的運算子;
              }
              break;
  case('+') : /* x 爲運算子 */
  case('-') :
  case('*') :
  case('/') :
```

4.3.3 運算式的轉換

```
        case('^') : y = 堆疊頂端的運算子;    /* 並未彈出堆疊頂端的資料 */
                    while(y 的優先順序高於或等於 x 的優先順序){
                       彈出堆疊頂端的運算子，並輸出至後置式;
                       y = 堆疊頂端的運算子; /* 並未彈出堆疊頂端的資料 */
                    }
                    將 x 放入堆疊;
                    break;
        default : /* x 爲運算元 */
                  直接將 x 輸出至後置式;
                  break;
        }
    }
```

步驟 2：將殘存於堆疊的運算子一一彈出，並輸出到後置式。

步驟 3：印出後置式。

4.3.3 運算式的轉換

【例３】利用堆疊將中置運算式轉換成後置運算式

表 4.3 利用堆疊將中置運算式 a+b/(c-d)*e-f 轉換成後置運算式的過程

步驟	讀入元素 x	處理方式	堆疊 	後置式的輸出結果 
0			空堆疊	
1	a	輸出 a	空堆疊	a
2	+	將+放入堆疊	+	a
3	b	輸出 b	+	ab
4	/	將/放入堆疊	+/	ab
5	(	將(放入堆疊	+/(	ab
6	c	輸出 c	+/(	abc

4.3.4 運算式的求值

表 4.3 利用堆疊將中置運算式 a+b/(c-d)*e-f 轉換成後置運算式的過程

步驟	讀入元素 x	處理方式	堆疊 (top)	後置式的輸出結果 →
7	-	將-放入堆疊	+/(-	abc
8	d	輸出 d	+/(-	abcd
9	)	從堆疊取出-並輸出 從堆疊取出(	+/	abcd-
10	*	將*放入堆疊	+*	abcd-/
11	e	輸出 e	+*	abcd-/e
12	-	從堆疊取出*並輸出 從堆疊取出+並輸出 再將-放入堆疊	-	abcd-/e*+
13	f	輸出 f	-	abcd-/e*+f
14		從堆疊取出-並輸出	空堆疊	abcd-/e*+f-

4.4 佇列 (Queue)

- 佇列具有以下特徵：
  1. 佇列是一個有順序性的資料結構。
  2. 「新增」時，資料從「入口」進入佇列；「刪除」時，離「出口」最近的資料先被刪除。
  3. 具有「先進先出 (First In First Out ： FIFO) 」(或「後進後出」 (Last In Last Out ： LILO)) 的特性。

4.4 佇列 (Queue)

- 佇列的實例
  - 高速公路收費站等待繳費的車龍。
  - 大學入學指定科目考試排隊量體溫的考生。
  - 戲院售票口排隊買票的人潮。

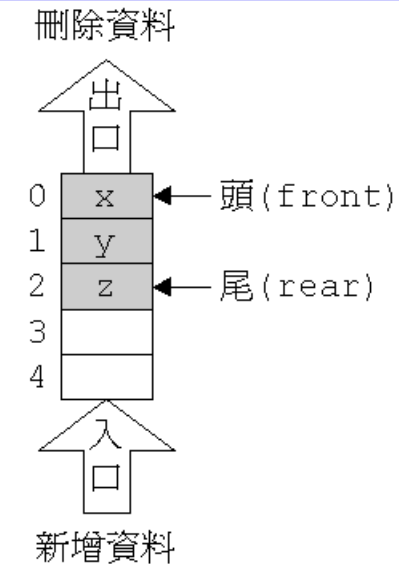


圖 4.10 佇列

4.4.1 以陣列實作佇列

- 以「陣列」實作佇列時可歸納成底下幾個基本操作：
  1. 宣告陣列 queue[n] ：
     陣列大小 n 即代表佇列容器的容量，且索引編號為 0、 1、 2、 ...、 n-1。
  2. 設定佇列的頭指標 (front) 和尾指標 (rear) ：
     一開始 front = rear = -1 ，表示佇列裡空無一物。請注意！我們將 front 指標指向佇列中排隊排在最前面一筆資料的前一號位置，而 rear 指標則指向佇列最後一筆資料之位置。

4.4.1 以陣列實作佇列

3. 判斷佇列是否已滿：
   當 rear = n-1 時表示佇列已滿。
4. 判斷佇列是否已空：
   當 front = rear 時表示佇列已空。
5. 新增資料：
   若佇列未滿，則 rear = rear + 1 後再將資料存入 queue [rear] 位置。
6. 刪除資料：
   若非空佇列，則 front = front + 1 ，然後自佇列將資料 queue [front] 取出。

4.4.1 以陣列實作佇列

【例 1 】以陣列來實作佇列的新增和刪除

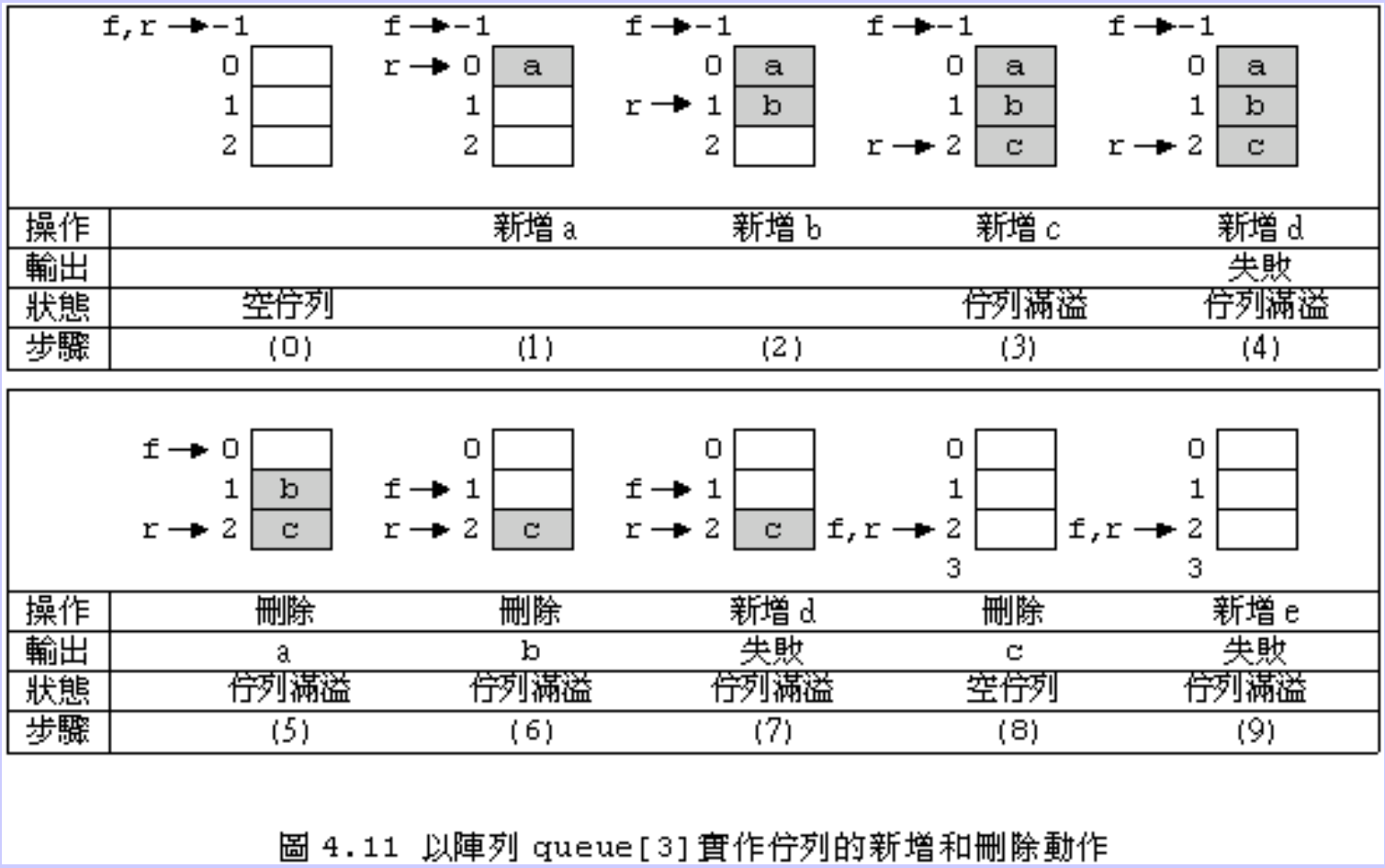


圖 4.11 以陣列 queue[3] 實作佇列的新增和刪除動作

4.4.2 以鏈結串列實作佇列

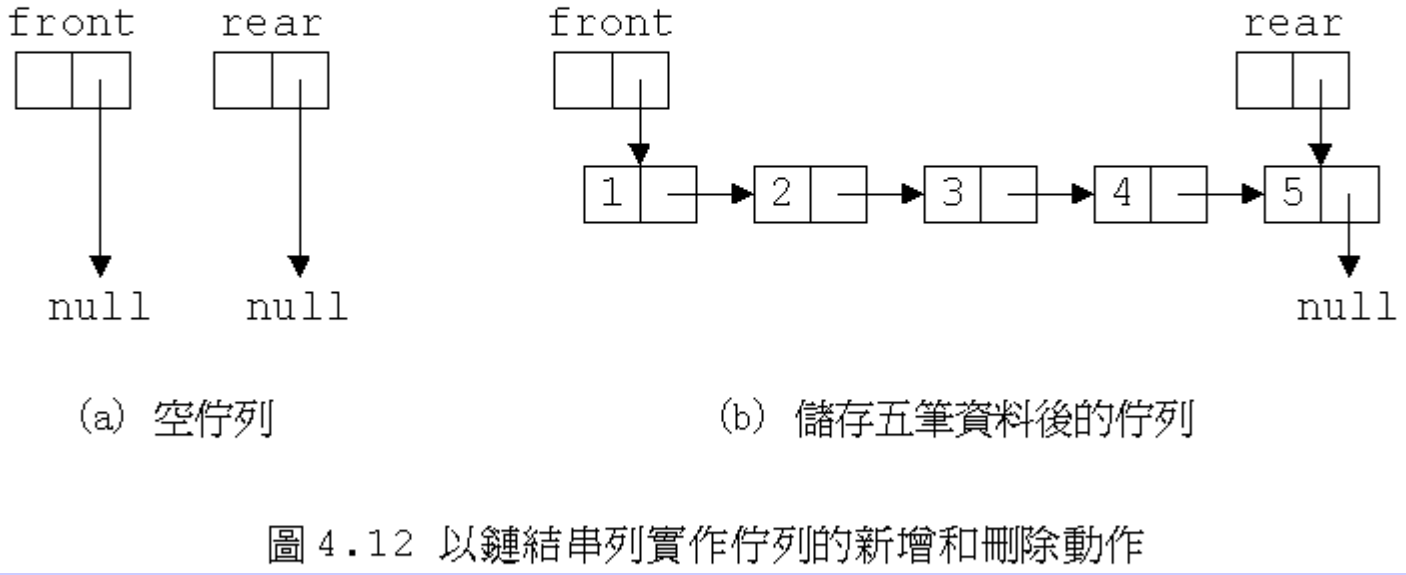


圖 4.12 以鏈結串列實作佇列的新增和刪除動作

4.4.2 以鏈結串列實作佇列

- 以「鏈結串列」實作佇列時可歸納成底下幾個基本操作：
  1. 宣告一個鏈結串列結構，一開始是一個空佇列，有一個頭節點 front 和一個尾節點 rear，且這兩個節點指標都指向 null，即 front=rear=null。
  2. 判斷佇列是否已空：
     當 front 節點指向 null 時，表示佇列已空無一物。

4.4.2 以鏈結串列實作佇列

3. 新增資料：
   首先，向記憶體索取一個節點空間 x ，然後將要新增的資料存入 x 的資料欄，將 x 的鏈結欄指向 null。若此時為空佇列則 front 和 rear 都指向 x。否則，令 rear 所指的節點為 y ，將 y 指向 x ， rear 亦指向 x。
4. 刪除資料：
   若非空佇列，則令 front 所指的節點為 x ，且 x 所指的節點為 y。輸出 x 的資料欄，再將 front 指向 y。

4.4.2 以鏈結串列實作佇列

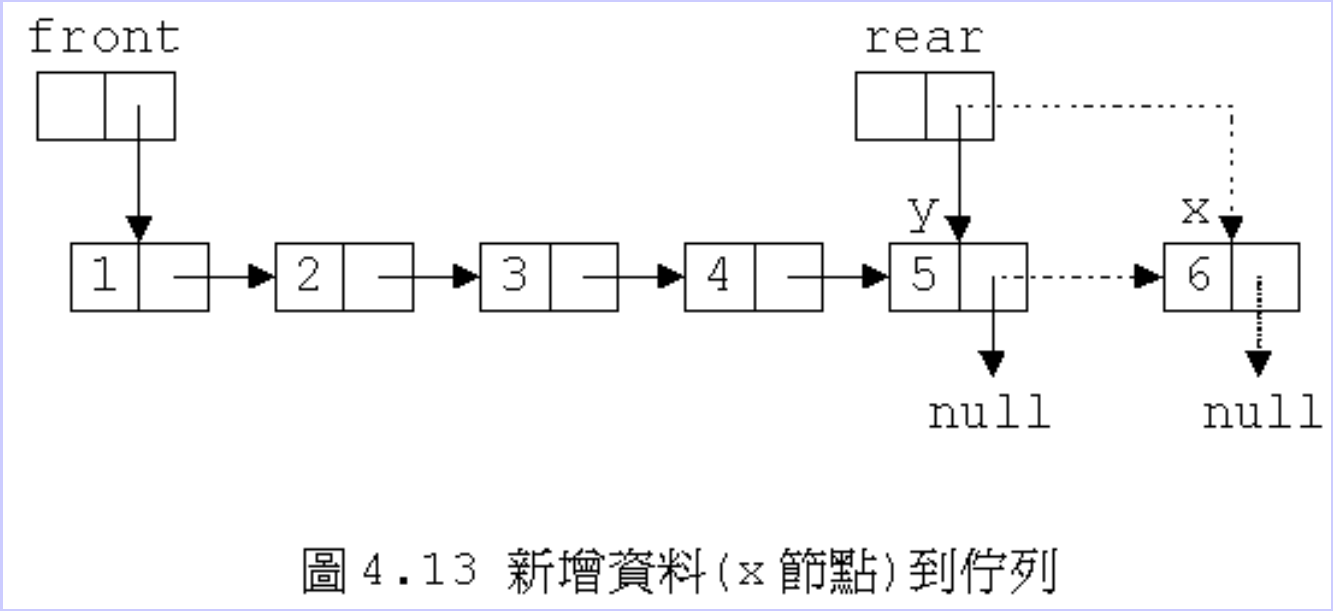


圖 4.13 新增資料(x 節點)到佇列

4.4.2 以鏈結串列實作佇列

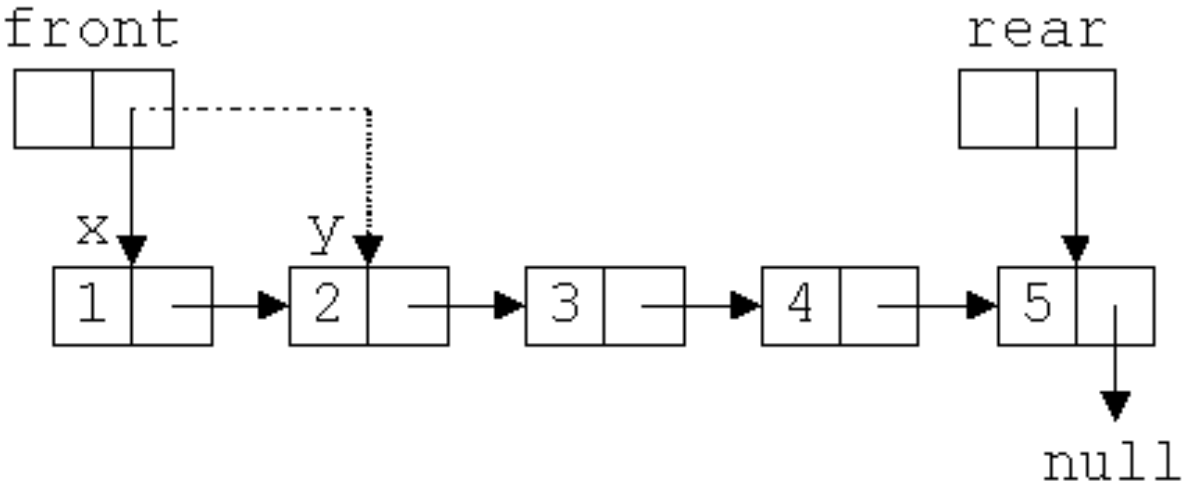


圖 4.14 從佇列刪除資料(x 節點)

4.4.3 佇列的應用

- 作業系統 (Operating System ， OS) 的工作 (Job) 排程
- 優先佇列

4.5 以陣列／鏈結串列實作堆疊／佇列之比較

表4.4 以「陣列」、「鏈結串列」實作「堆疊」之比較

堆疊	以「陣列」實作堆疊	以「鏈結串列」實作堆疊
宣告	陣列	節點結構
記憶體空間	宣告時一次索取 (Compile time Allocate Memory)	須要時才索取記憶體空間(Run time Allocate Memory)
新增資料	速度快	須索取記憶體，速度慢
刪除資料	速度快	須歸還記憶體，速度慢
優點	● 程式容易撰寫 ● 執行速度較快	● 可對記憶體做有效運用 ● 不容易發生滿溢
缺點	● 宣告太多而用太少時會浪費記憶體 ● 容易發生滿溢	● 程式較不易撰寫 ● 執行速度較慢 ● 鏈結斷裂時資料將流失

4.5 以陣列／鏈結串列實作堆疊／佇列之比較

表 4.5 以「陣列」、「鏈結串列」實作「佇列」之比較

佇列	以「陣列」實作佇列	以「鏈結串列」實作佇列
宣告	陣列	節點結構
記憶體空間	宣告時一次索取 (Compile time Allocate Memory)	須要時才索取記憶體空間(Run time Allocate Memory)
新增資料	速度快	須索取記憶體，速度慢
刪除資料	● 速度快 ● 刪除後空下來的位置不能再使用，造成記憶體的浪費	須歸還記憶體，速度慢
優點	● 程式容易撰寫 ● 執行速度較快	● 可對記憶體做有效運用 ● 不容易發生滿溢
缺點	● 宣告太多而用太少時會浪費記憶體 ● 刪除資料後空下來的位置不能再使用，造成記憶體的浪費 ● 容易發生滿溢	● 程式較不易撰寫 ● 執行速度較慢 ● 鏈結斷裂時資料將流失

4.6.1 多重堆疊 (Multiple Stacks)

- 用陣列來實作堆疊或佇列可能產生兩個主要缺點：
  1. 宣告太多而使用太少，造成記憶體的浪費。
  2. 當資料太多而宣告量不足時，造成堆疊滿溢 (Overflow)。
- 多重佇列是採用「共用單一陣列空間」的技巧來對記憶體做更有效之應用。

4.6.1 多重堆疊 (Multiple Stacks)

- 堆疊空了：
  當 t[i]=b[i] 時，表示第 i 號堆疊已經空了。
- 堆疊滿溢：
  當 t[i]=b[i+1] 時，表示第 i 號堆疊已經滿溢

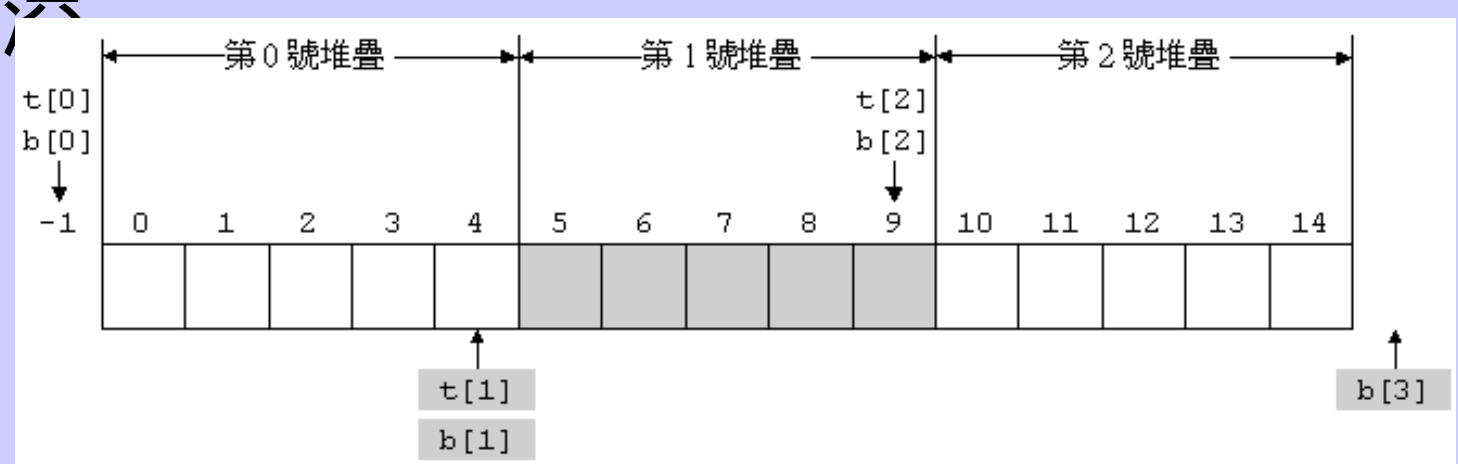


圖 4.15 多重堆疊(空堆疊)

4.6.1 多重堆疊 (Multiple Stacks)

- 第 1 號堆疊已滿

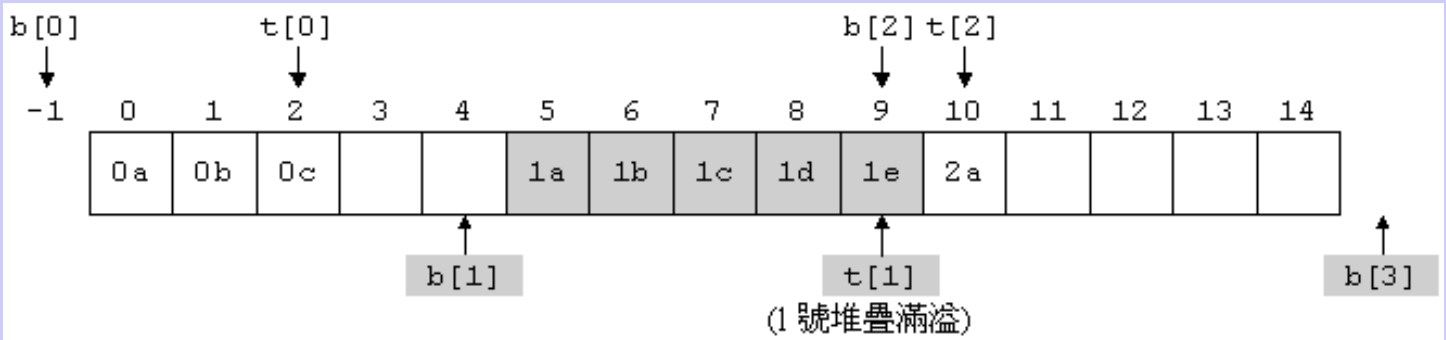


圖 4.16 多重堆疊(第 1 號堆疊已經滿溢(t[1]=b[2]=9))

4.6.1 多重堆疊 (Multiple Stacks)

- 重分配後便可將資料 1f 存放至 stack[t[1]]=stack[10] 號位置。

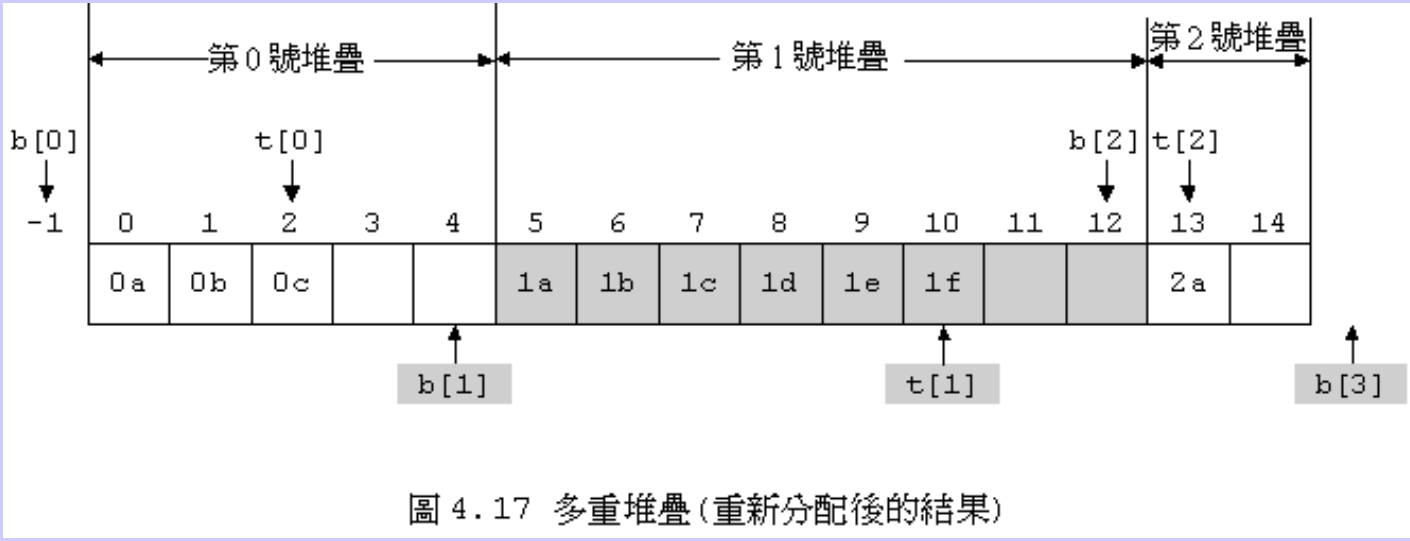


圖 4.17 多重堆疊(重新分配後的結果)

4.6.2 多重佇列 (Multiple Queues)

- 佇列滿溢：
  當 r[i]=b[i] 時，表示第 i 號佇列已經滿溢。
- 佇列空了：
  當 f[i]=r[i] 時，表示第 i 號佇列已經空了。

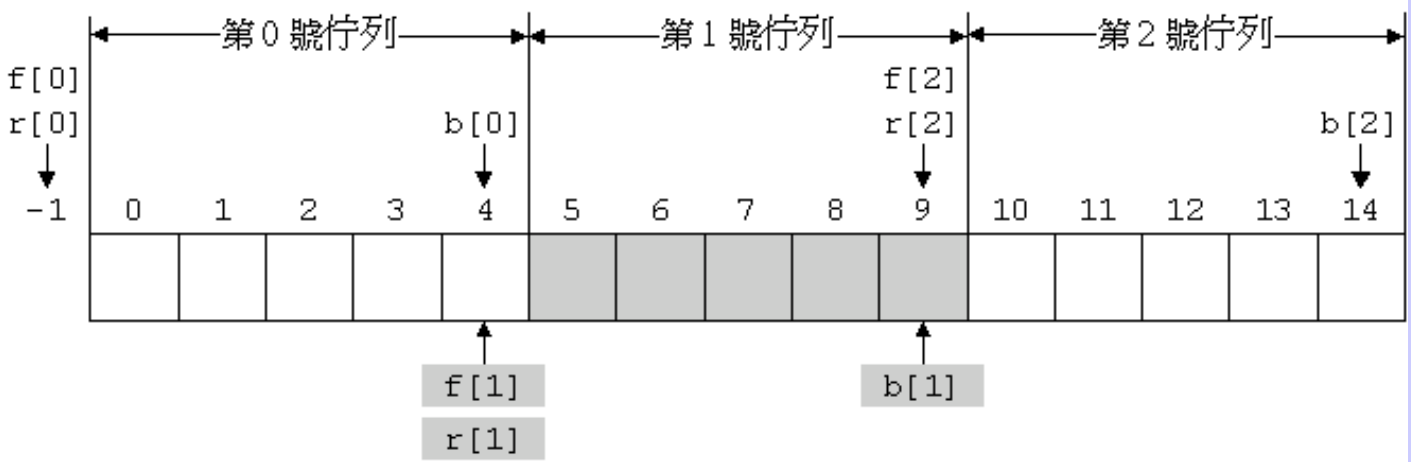


圖 4.18 多重佇列(空佇列)

4.6.2 多重佇列 (Multiple Queues)

- 第 1 號佇列已經滿溢 (r[1]=b[1]=9) 。

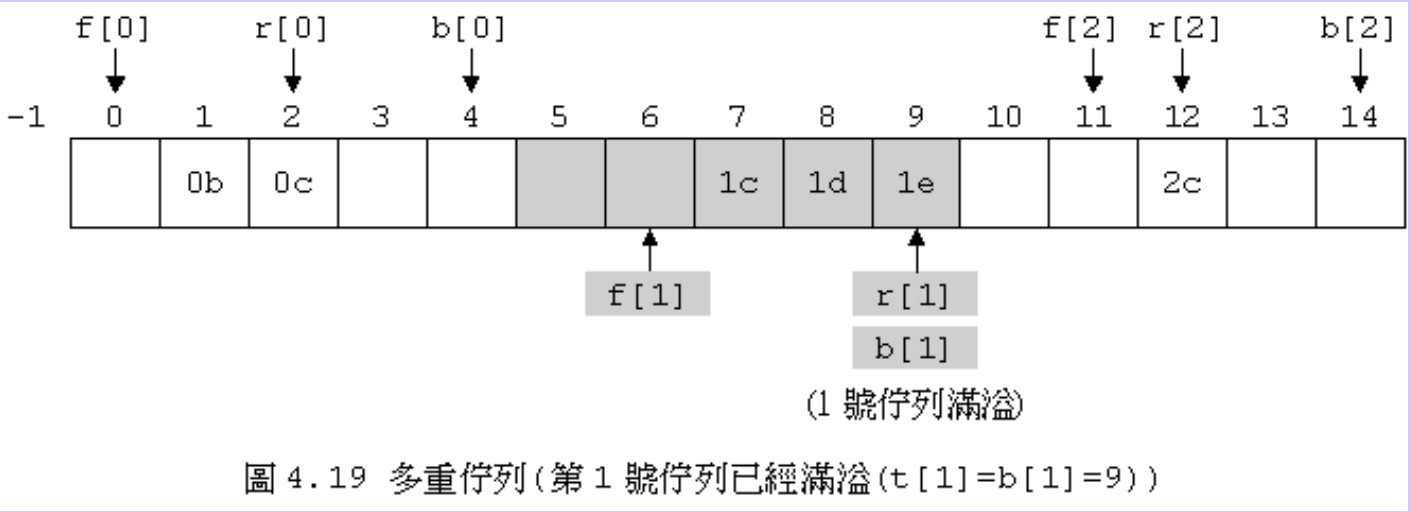


圖 4.19 多重佇列(第 1 號佇列已經滿溢(t[1]=b[1]=9))

4.6.2 多重佇列 (Multiple Queues)

- 重分配後空出 queue[10] 和 queue[11] ，便可供第 1 號佇列使用

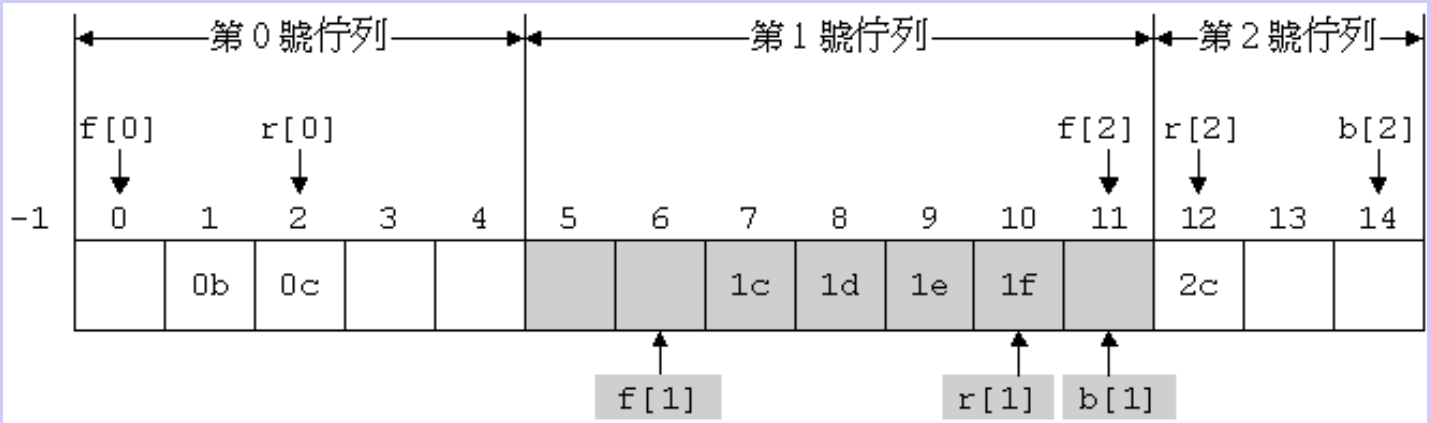


圖 4.20 多重佇列(重新分配後的結果)

4.6.3 優先佇列 (Priority Queues)

- 優先佇列–以陣列實作
  - 有一個頭指標 f 用來指向整個佇列的頭端。
  - 有三個尾指標 r_a、 r_b 和 r_c 則分別指向 A、 B、 C 級佇列的尾端。
  - r_a 也是 B 級的頭端， r_b 也是 C 級的頭端。

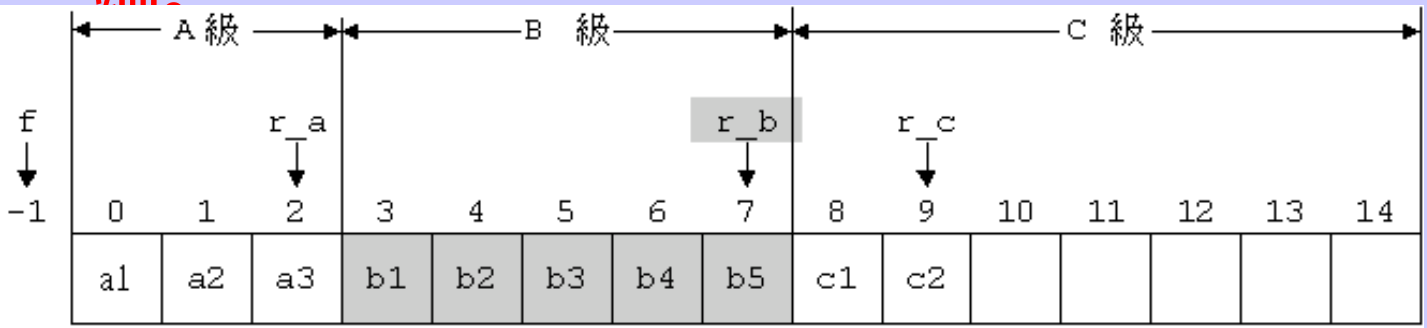


圖 4.21 優先佇列(以陣列實作)

4.6.3 優先佇列 (Priority Queues)

- 優先佇列
  - 假設有一屬於 B 級的資料 b6 要加入佇列，，但 queue[8] 已經被 C 級佇列所佔用，因此須：
    1. 將 c2 搬移到 queue[10] ，再將 r_c 改成 10。
    2. 將 c1 搬移到 queue[9]。

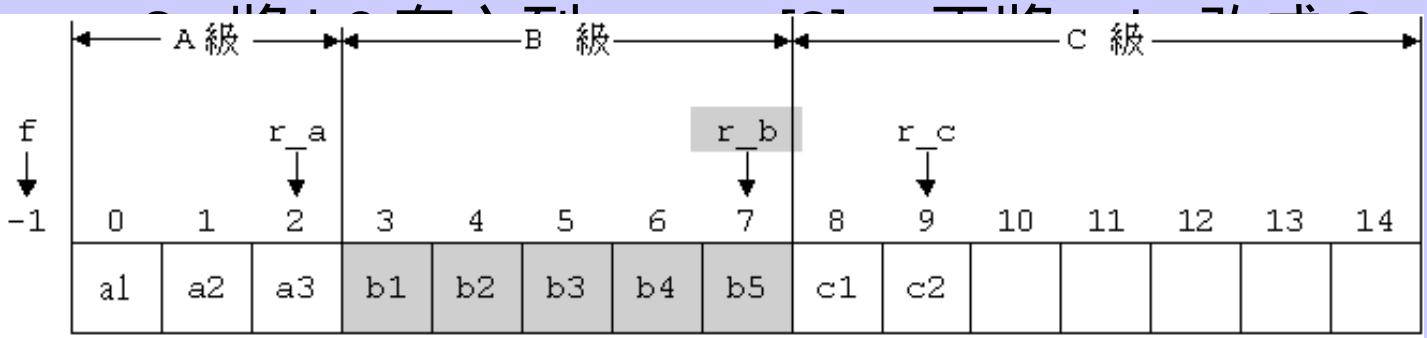


圖 4.21 優先佇列(以陣列實作)

4.6.3 優先佇列 (Priority Queues)

- 優先佇列
  - 若要刪除 B 級佇列的資料，則 b1 須被刪除。一旦刪除了 b1 ，排在它後面的 b2 、 b3 、 b4 、 b5 和、 c1 、 c2 都須往前（左）移動

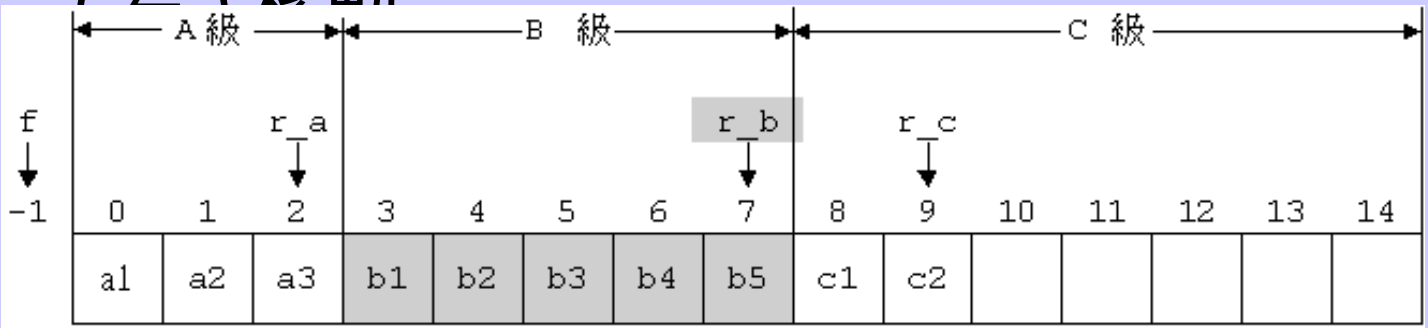


圖 4.21 優先佇列(以陣列實作)

4.6.3 優先佇列 (Priority Queues)

- 優先佇列–以鏈結串列實作

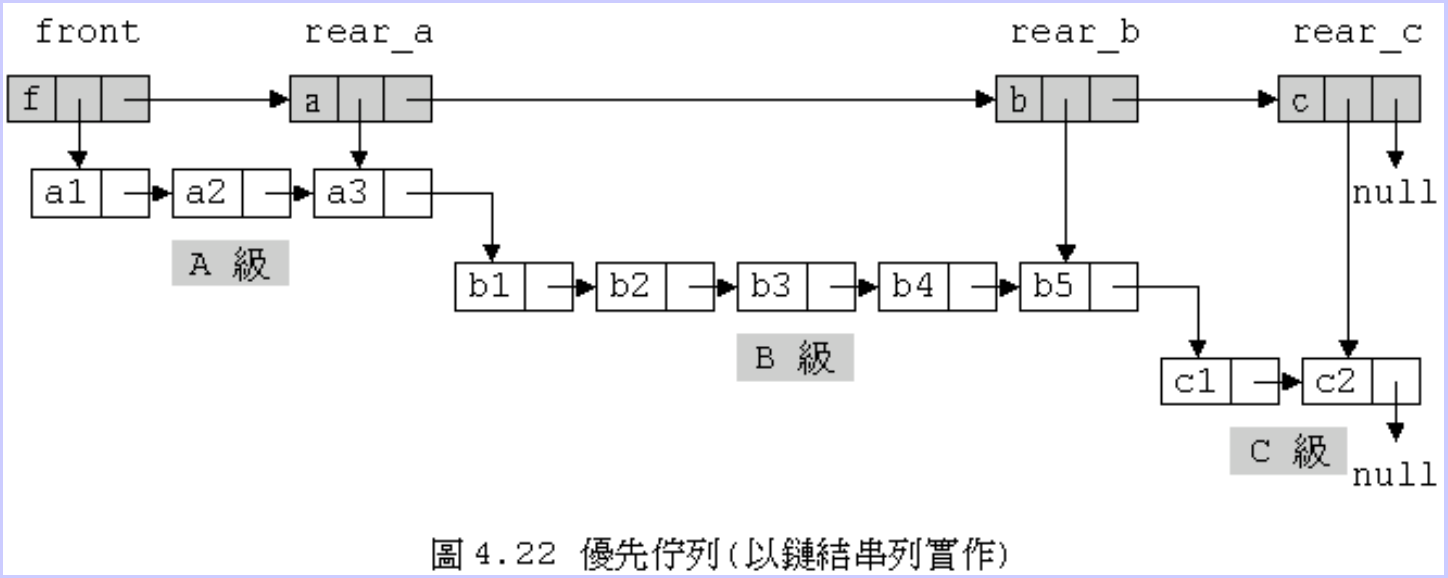


圖 4.22 優先佇列(以鏈結串列實作)

4.6.3 優先佇列 (Priority Queues)

- 優先佇列–以鏈結串列實作
  - 加入 b6 的方法為：
    1. 向記憶體索取一個節點，令其為 x。將 b6 儲存到 x 的資料欄。
    2. 沿著 front 找到 rear_b ，令 rear_b 的 down 鏈結所指之節點為 y。
    3. 令 y 的鏈結所指之節點為 z。
    4. 將 x 指向 z ，將 y 指向 x。
    5. 將 rear_b 之 down 鏈結指向 x。

4.6.3 優先佇列 (Priority Queues)

- 優先佇列–以鏈結串列實作
  - 刪除 b1 的方法為：
    1. 沿著 front 找到 rear_a ，令 rear_a 的 down 鏈結所指之節點為 x 。
    2. 令 x 的鏈結所指之節點為 y 。
    3. 令 y 的鏈結所指之節點為 z 。
    4. 將 x 指向 z 。
    5. 將 y 歸還給記憶體串列。

4.6.4 環狀佇列 (Circular Queues)

- 以「陣列」實作「環狀佇列」時可歸納成底下幾個基本操作：

  1. 宣告陣列 queue[n] ：
     陣列大小 n 即代表佇列容器的容量，且索引編號為 0、 1、 2、 ...、 n-1。
  2. 設定佇列的頭指標 (front) 和尾指標 (rear) ：

     一開始佇列裡空無一物， front = rear = n-1 ，而非 -1。我們仍然將 front 指標指向佇列中排隊排在最前面一筆資料的前一號位置，而 rear 指標則指向佇列最後一筆資料之位置。

4.6.4 環狀佇列 (Circular Queues)

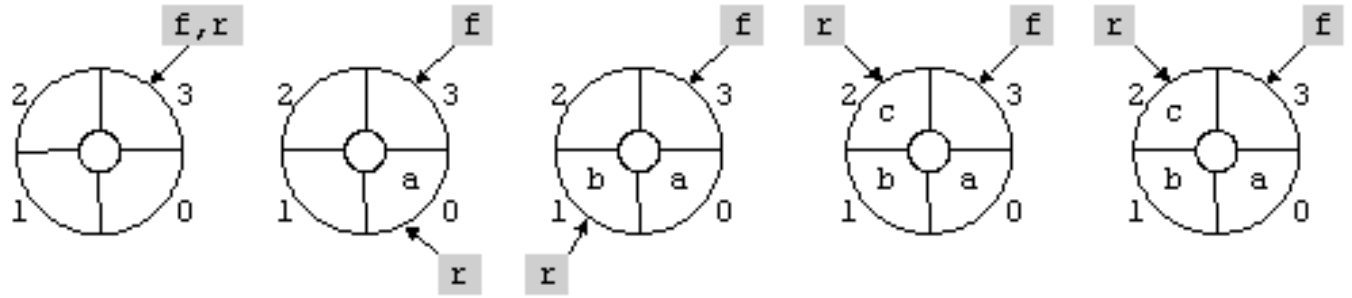
4. 判斷佇列是否已滿：
   當 front=(rear+1) mod n 時，表示佇列已滿。
5. 判斷佇列是否已空：
   當 front=rear 時，表示佇列已空。
6. 新增資料：
   若佇列未滿，則 rear=(rear+1) mod n 後，再將資料存入 queue[rear] 位置。
7. 刪除資料：
   若非空佇列，則 front=(front+1) mod n ，然後自佇列將資料 queue[front] 取出。

4.6.4 環狀佇列 (Circular Queues)

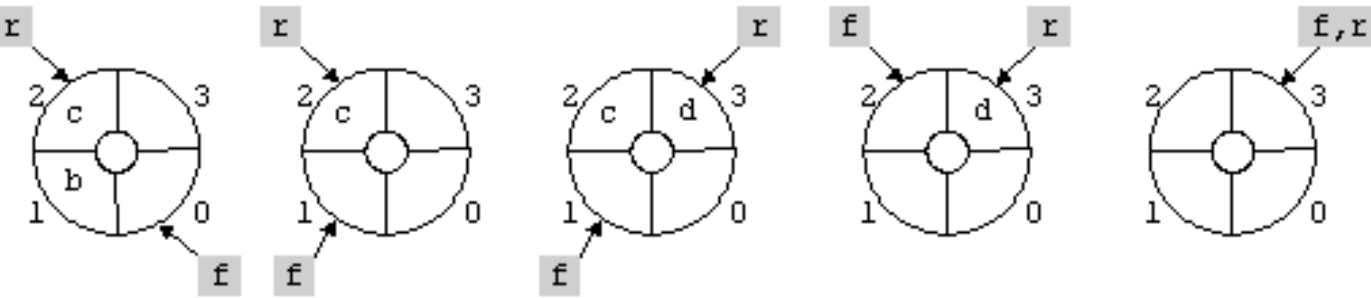
【例 1 】以陣列來實作環狀佇列的新增和刪除

【解答】假設有一個陣列 queue[4](即 n=4) ，我們運用 queue 陣列來模擬環狀佇列的運作過程如圖 4.24 所示。為方便起見，我們用 f 來代表 front 指標，用 r 來代表 rear 指標。

4.6.4 環狀佇列 (Circular Queues)



操作		新增 a	新增 b	新增 c	新增 d
輸出					失敗
狀態	空佇列			佇列滿溢	佇列滿溢
步驟	(0)	(1)	(2)	(3)	(4)



操作	刪除	刪除	新增 d	刪除	刪除
輸出	a	b		c	c
狀態					空佇列
步驟	(5)	(6)	(7)	(8)	(9)

圖 4.24 環狀佇列的新增和刪除

4.6.4 環狀佇列 (Circular Queues)

- 環狀佇列的幾個主要特徵：
  1. 環狀佇列乃用「陣列」來實作。
  2. 環狀佇列之功能在對記憶體做更有效之應用。
  3. 環狀佇列不須搬移資料。
  4. 環狀佇列 queue[n] 最多能存放 n-1 筆資料。
  5. 環狀佇列資料被刪除後所空下來的位置可以再利用。

4.6.4 環狀佇列 (Circular Queues)

表 4.6 以「陣列」實作「環狀佇列」及「一般佇列」之比較

	環狀佇列	一般佇列
陣列宣告	q[n]	q[n]
陣列元素	q[0]、q[1]、…、q[n-1]	q[0]、q[1]、…、q[n-1]
最大容量	n-1	n
初值	front=rear=n-1	front=rear=-1
佇列滿溢	front=(rear+1) mod n	rear=n-1
空佇列	front=rear	front=rear
新增資料的位置	rear=(rear+1) mod n	rear=rear+1
刪除資料的位置	front=(front+1) mod n	front=front+1
佇列中僅剩一個元素	rear=(front+1) mod n	rear=front+1

4.6.5 雙向佇列 (Double Ended Queues)

- 雙向佇列 (簡稱 Deque)
  - f[0] 用來指向左邊佇列的頭。
  - r[0] 用來指向左邊佇列的尾。
  - f[1] 用來指向右邊佇列的頭。
  - r[1] 用來指向右邊佇列的尾。

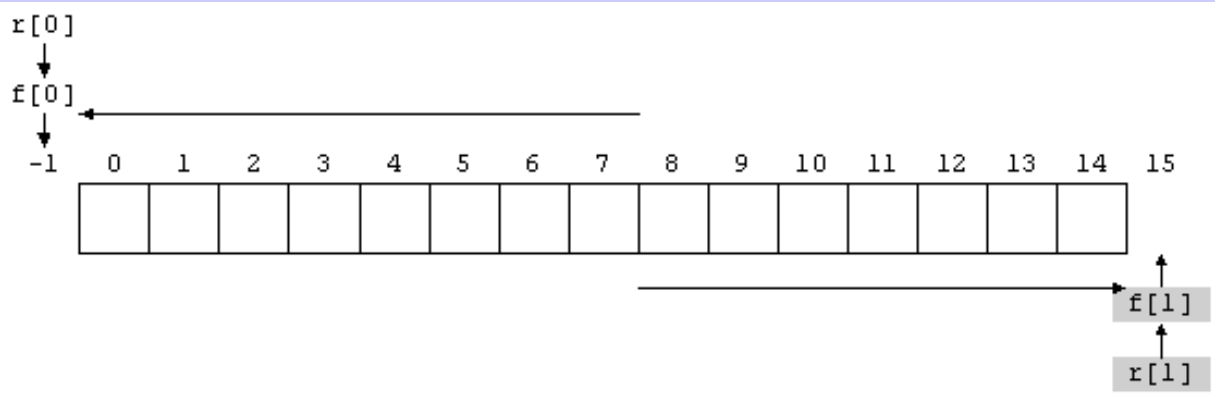


圖 4.25 雙向佇列(空佇列)

4.6.5 雙向佇列 (Double Ended Queues)

- 雙向佇列 (簡稱 Deque)
  - 在左邊佇列依序新增 0a 、 0b 和 0c
  - 在右邊佇列依序新增 1a 、 1b 、 1c 、 1d 和 1e

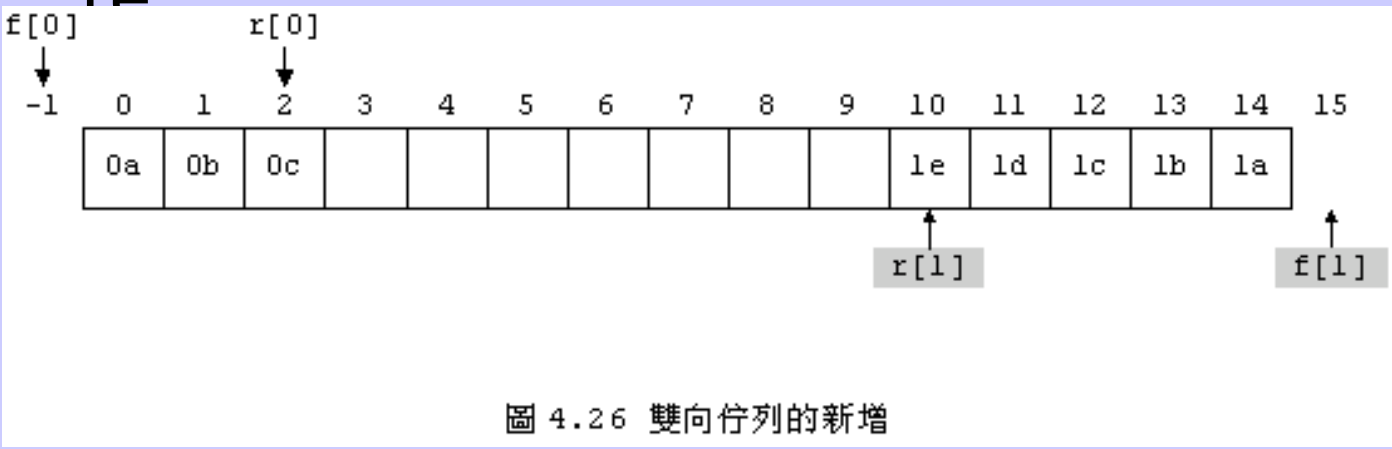


圖 4.26 雙向佇列的新增

4.6.5 雙向佇列 (Double Ended Queues)

- 雙向佇列 (簡稱 Deque)
  - 從左邊佇列刪除一筆資料
  - 從右邊佇列刪除兩筆資料



圖 4.27 雙向佇列的刪除

4.6.5 雙向佇列 (Double Ended Queues)

【例 1 】何種情況下表示雙向佇列已滿溢？

【解答】

- 當 r[0]+1=r[1] 時，表示雙向佇列已滿溢。

4.6.5 雙向佇列 (Double Ended Queues)

【例 2 】何種情況下表示雙向佇列為一空佇列？

【解答】

- 當 f[0]=r[0] 時，表示雙向佇列的左邊為一空佇列。
- 當 f[1]=r[1] 時，表示雙向佇列的右邊為一空佇列。
- 當 f[0]=r[0] 且 f[0]=r[0] 時，表示整個雙向佇列為一空佇列。

4.6.5 雙向佇列 (Double Ended Queues)

【例 3 】試說明新增資料到雙向佇列的流程？

【解答】

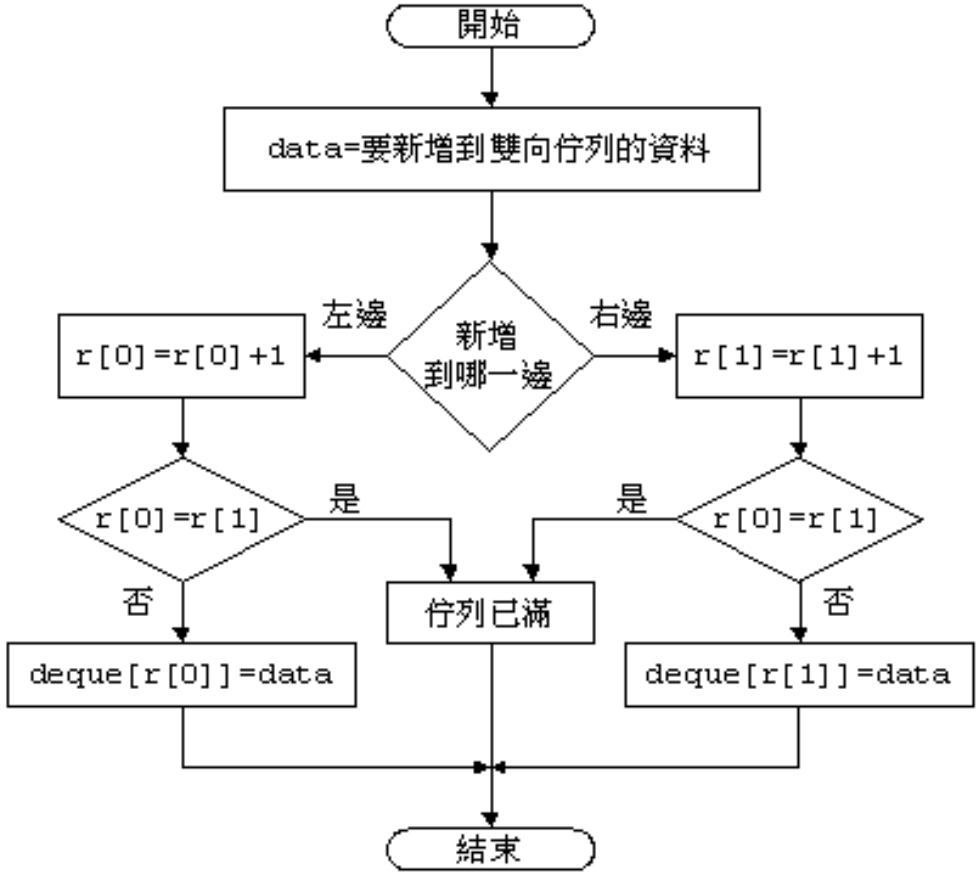


圖 4.28 雙向佇列的新增資料流程

4.6.5 雙向佇列 (Double Ended Queues)

【例 4 】試說明從雙向佇列刪除一筆資料的流程？

【解答】

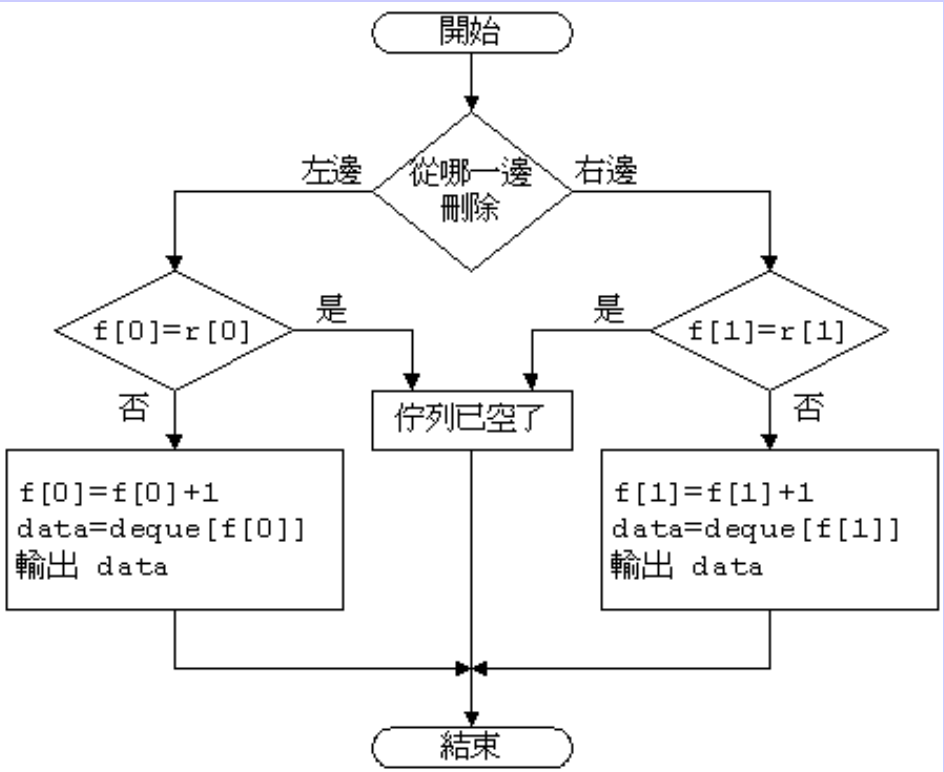


圖 4.29 雙向佇列的刪除資料流程